Course Name:

Advanced Programming (OOP)

Institute of Engineering and Management

Vehicle Rental Management System

Submitted by:

Student Name: Apurba Mondal

Roll Number: 59

Enrollment No: 12024052017061

Instructor: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

Spring 2026

1. Problem Statement

Traditional vehicle rental businesses rely heavily on manual processes — paper-based records, phone-based availability checks, and handwritten receipts — which are error-prone, slow, and difficult to scale. There is a pressing need for a software solution that automates core rental operations while remaining simple enough for front-desk staff to use without technical training.

This project addresses that need by developing a Vehicle Rental Management System, a desktop application that allows staff to manage a fleet of vehicles (cars, trucks, and motorcycles), register customers, process rentals, handle returns, and view live availability — all through a graphical user interface.

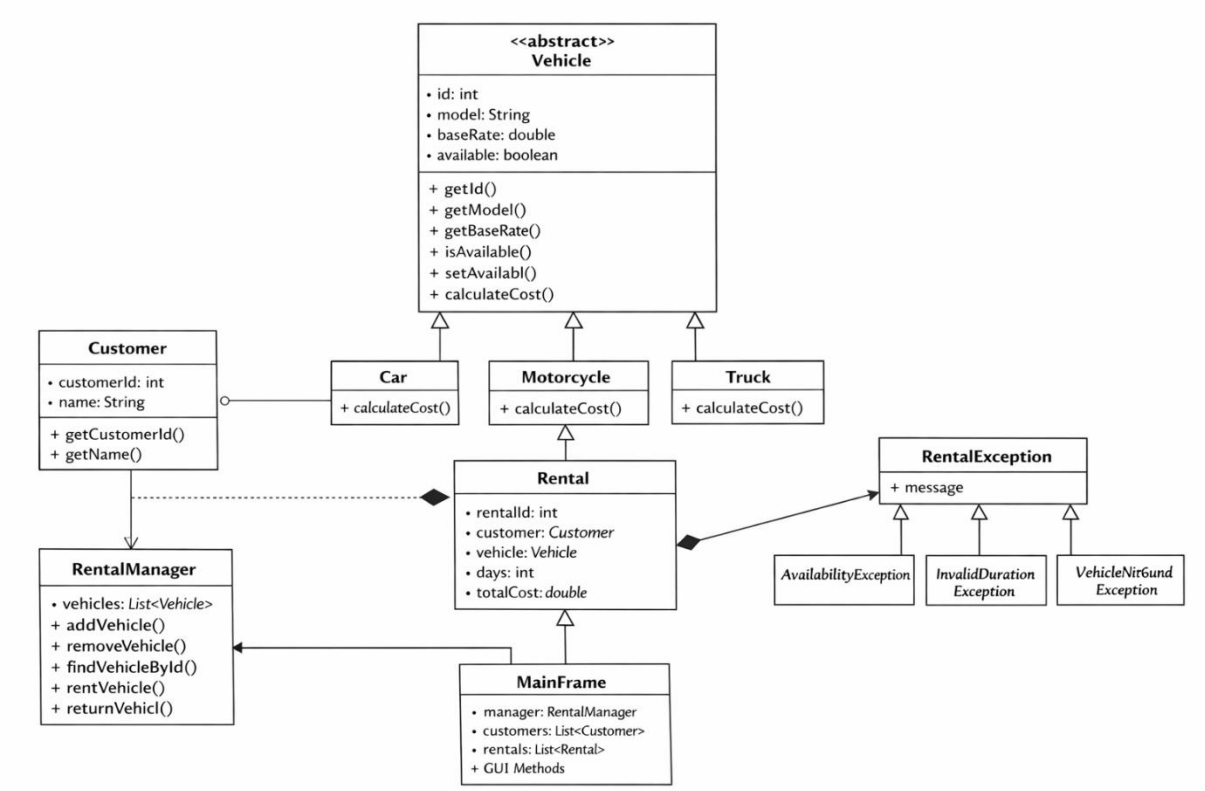
The system must satisfy the following requirements:

- Maintain an in-memory fleet of vehicles categorised by type (Car, Truck, Motorcycle), each with a unique ID, model name, base rate, and availability status.
- Allow new vehicles to be added to or removed from the fleet at runtime.
- Register customers with unique IDs and names, and prevent duplicate registrations.
- Process a rental by linking a customer to a vehicle for a specified number of days, computing the total cost according to type-specific pricing rules, and marking the vehicle as unavailable.
- Handle vehicle returns by restoring availability so the vehicle can be rented again.
- Provide a real-time status panel listing every vehicle alongside its type, daily rate, and current availability.
- Raise meaningful, typed exceptions for invalid inputs such as non-positive rental durations, unknown vehicle IDs, or attempts to rent an already-rented vehicle.

The solution is implemented entirely in Java, applying object-oriented principles including inheritance, polymorphism, encapsulation, and abstraction, as well as a custom exception hierarchy and a Swing-based GUI.

2. System Design

UML Class Diagram 2.1



Description of Major Classes

Vehicle (abstract): The root of the vehicle hierarchy. Holds common fields — id, model, baseRate, and available — and declares the abstract method calculateCost(int days), which each subclass overrides.

Car, Truck, Motorcycle: Concrete subclasses of Vehicle. Each overrides calculateCost() with its own pricing logic: Car charges baseRate × days; Motorcycle applies a 10 % discount; Truck adds a flat Rs. 100 surcharge per rental.

Customer: A simple domain object holding a customer's ID and name. Instances are stored in an ArrayList inside MainFrame.

Rental: An immutable record that associates a Customer with a Vehicle, the number of rental days, and the computed total cost. Rental objects are appended to a list each time a successful rental is processed.

RentalManager (service): The central service class. It owns the in-memory vehicle list and exposes addVehicle, removeVehicle, findVehicleById, getAllVehicles, rentVehicle, and returnVehicle. It validates inputs and throws typed exceptions.

MainFrame (GUI): A JFrame subclass that wires together the entire user interface using a CardLayout with five panels: Edit Fleet, Add Customer, Rent a Car, Return Vehicle, and Car Status & Rates.

Exception classes: RentalException (base), AvailabilityException, InvalidDurationException, and VehicleNotFoundException form a typed exception hierarchy used throughout the service layer.

Package Structure

```
src/
  domain/
    Vehicle.java          (abstract base class)
    Car.java
    Truck.java
    Motorcycle.java
    Customer.java
    Rental.java
  service/
    RentalManager.java    (business logic)
  exception/
    RentalException.java
    AvailabilityException.java
    InvalidDurationException.java
    VehicleNotFoundException.java
  gui/
    MainFrame.java        (Swing GUI)
  main/
    GUIMain.java          (entry point)
```

Interaction Flow

When the application starts, GUIMain invokes MainFrame on the Swing Event Dispatch Thread. The constructor of MainFrame instantiates RentalManager and populates it with six default vehicles. The user then interacts with the menu on the left side of the window, which switches the central CardLayout panel to the appropriate view.

For a typical rental flow: the staff member first navigates to Add Customer to register a customer, then switches to Rent a Car, enters the customer ID, vehicle ID, and number of days, and clicks Confirm Rental. MainFrame calls RentalManager.rentVehicle(), which validates the input, marks the vehicle unavailable, and returns the total cost. A Rental object is created and appended to the rentals list. The output log at the bottom of the window displays a confirmation message. On return, the staff member uses the Return Vehicle panel, which calls RentalManager.returnVehicle() and restores availability.

3. OOP Concepts Demonstrated

Inheritance

Where: Car, Truck, and Motorcycle all extend the abstract class Vehicle.

Why: Inheritance eliminates code duplication for shared fields (id, model, baseRate, available) and allows all three vehicle types to be stored and manipulated through a single Vehicle reference.

```
// Vehicle.java
public abstract class Vehicle {
    protected int id;
    protected String model;
    protected double baseRate;
    protected boolean available;

    public abstract double calculateCost(int days);
}

// Car.java
public class Car extends Vehicle {
    @Override
    public double calculateCost(int days) {
        return baseRate * days;
    }
}
```

Car inherits all fields from Vehicle and only provides its own cost formula, keeping the subclass concise.

Polymorphism

Where: RentalManager stores all vehicles as List<Vehicle> and calls calculateCost() without knowing the concrete type.

Why: Polymorphism lets the system apply the correct pricing rule at runtime without if-else chains, making it easy to add new vehicle types in the future.

```
// RentalManager.java
public double rentVehicle(int vehicleId, int days)
        throws RentalException {
    if (days <= 0) throw new InvalidDurationException();
    Vehicle vehicle = findVehicleById(vehicleId);
    if (vehicle == null) throw new VehicleNotFoundException();
    if (!vehicle.isAvailable()) throw new AvailabilityException();
    vehicle.setAvailable(false);
    return vehicle.calculateCost(days); // dispatched polymorphically
}
```

The call to calculateCost() resolves to Car, Truck, or Motorcycle's implementation at runtime depending on the actual object type.

Abstraction

Where: Vehicle is declared abstract with the abstract method calculateCost().

Why: Abstraction hides the pricing details from the service layer. RentalManager works only with the abstract contract; the specifics of how each type calculates cost are encapsulated inside the subclass.

```
public abstract class Vehicle {
    // Common state hidden from external classes
    protected double baseRate;

    // Contract — subclasses must implement this
    public abstract double calculateCost(int days);
}
```

Encapsulation

Where: All fields in Vehicle, Customer, and Rental are private or protected. Access is only through public getters and setters.

Why: Encapsulation prevents external code from setting an invalid state directly — for example, marking a vehicle available/unavailable is controlled only through setAvailable(), which is called by RentalManager after validating the operation.

```
// Vehicle.java
private boolean available;

public boolean isAvailable() { return available; }
public void setAvailable(boolean status) {
    this.available = status;
}
```

Custom Exception Hierarchy

Where: The exception package contains RentalException (base) and three specific subclasses.

Why: A typed hierarchy allows the GUI to catch the base type RentalException in a single catch block while still propagating meaningful messages to the user.

```
// RentalException.java (base)
public class RentalException extends Exception {
    public RentalException(String message) { super(message); }
}
```

```
// AvailabilityException.java
public class AvailabilityException extends RentalException {
    public AvailabilityException() {
        super("Vehicle already rented");
    }
}
```

4. Implementation Highlights

Type-Specific Cost Calculation

Each vehicle subclass overrides calculateCost() with its own pricing formula. This ensures the business rules are co-located with the type they describe rather than scattered through the service layer.

```
// Car.java  —  flat daily rate
public double calculateCost(int days) { return baseRate * days; }

// Motorcycle.java  —  10 % discount
public double calculateCost(int days) { return baseRate * days * 0.9; }

// Truck.java  —  flat Rs.100 surcharge
public double calculateCost(int days) { return (baseRate * days) + 100; }
```

Adding a new vehicle type (e.g., Van) requires only a new subclass with its own formula — no changes to RentalManager are needed.

Validated Rental Processing in RentalManager

rentVehicle() enforces all business rules before committing any state change. Exceptions are thrown in a guard-clause style, keeping the happy-path code at the end and easy to read.

```
public double rentVehicle(int vehicleId, int days)
        throws RentalException {
    if (days <= 0) throw new InvalidDurationException();
    Vehicle v = findVehicleById(vehicleId);
    if (v == null) throw new VehicleNotFoundException();
    if (!v.isAvailable()) throw new AvailabilityException();
    v.setAvailable(false);
    return v.calculateCost(days);
}
```

CardLayout-Based GUI Navigation

The five functional panels are pre-built and stored inside a CardLayout container. Switching views involves only a single show() call, with no panel re-creation overhead.

```
private JPanel buildCardPanel() {
    cardLayout = new CardLayout();
```

```
    mainPanel = new JPanel(cardLayout);
    mainPanel.add(buildFleetPanel(),    "FLEET");
    mainPanel.add(buildCustomerPanel(), "CUSTOMER");
    mainPanel.add(buildRentPanel(),     "RENT");
    mainPanel.add(buildStatusPanel(),   "STATUS");
    mainPanel.add(buildReturnPanel(),   "RETURN");
    return mainPanel;
}
```

Vehicle Return with Availability Restoration

returnVehicle() mirrors rentVehicle() by applying symmetric validation: it checks the vehicle exists and is currently rented before restoring availability, preventing double-returns.

```
public void returnVehicle(int vehicleId) throws RentalException {
    Vehicle v = findVehicleById(vehicleId);
    if (v == null) throw new VehicleNotFoundException();
    if (v.isAvailable())
        throw new RentalException("Vehicle is already available");
    v.setAvailable(true);
}
```

5. Testing & Error Handling

Test Cases Considered

#	Input / Action	Expected Result	Exception Thrown
1	Rent vehicle ID 1 for 3 days	Cost = Rs.150, status = Rented	None
2	Rent vehicle ID 1 again (already rented)	Error message displayed	AvailabilityException
3	Rent with days = 0	Error message displayed	InvalidDurationException
4	Rent vehicle ID 99 (not in fleet)	Error message displayed	VehicleNotFoundException
5	Return vehicle ID 1 after renting	Status restored to Available	None
6	Return vehicle ID 1 again (already available)	Error message displayed	RentalException
7	Add customer with duplicate ID	Duplicate warning shown	None (GUI check)
8	Rent without registering customer first	Customer-not-found message	None (GUI check)

Edge Cases Handled

- Zero or negative rental duration — caught by the days <= 0 guard in rentVehicle() and raises InvalidDurationException.

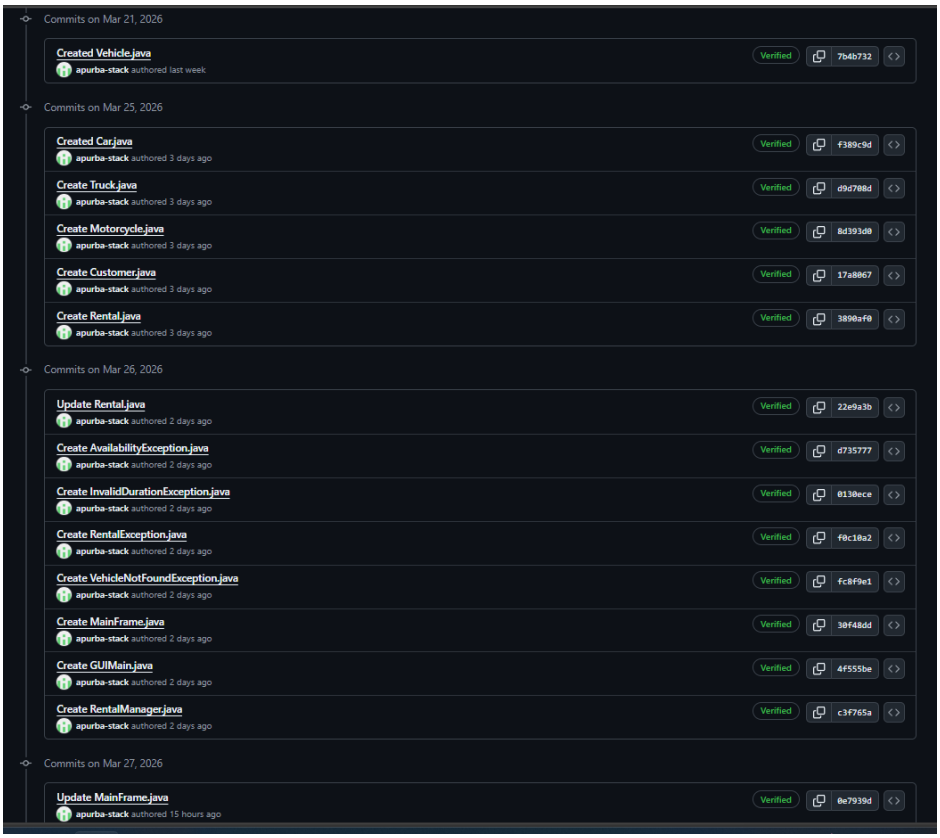
- Vehicle ID not present in the fleet — findVehicleById() returns null, triggering VehicleNotFoundException.
- Renting an already-rented vehicle — isAvailable() check raises AvailabilityException before any state is mutated.
- Returning a vehicle that was never rented — symmetric guard in returnVehicle() raises RentalException.
- Empty model name when adding a vehicle — validated in the GUI layer before calling addVehicle().
- Duplicate vehicle ID — checked in the GUI layer; addVehicle() is not called if the ID already exists.
- Duplicate customer ID — checked by iterating the customers list before adding.
- Non-numeric input in text fields — caught by NumberFormatException in every action listener.

Failure Scenarios

All exceptions surface as messages in the output log panel at the bottom of the window. No exception causes the application to crash. The try-catch structure in each action listener ensures that a failed operation leaves the vehicle list and rental list in the same state they were in before the attempt, since state changes (setAvailable, list.add) only occur after all validations pass.

6. Git Workflow

Commit History Screenshot



Development Progression 6.2

The project was developed iteratively across the following phases:

- Phase 1 — Domain layer: Defined the abstract Vehicle class and the three concrete subclasses (Car, Truck, Motorcycle) along with the Customer and Rental data classes.
- Phase 2 — Exception hierarchy: Created RentalException and its three typed subclasses to support meaningful error reporting.
- Phase 3 — Service layer: Implemented RentalManager with in-memory storage, full CRUD for vehicles, and validated rental/return operations.
- Phase 4 — GUI: Built MainFrame with CardLayout navigation, five functional panels, and an output log.
- Phase 5 — Cleanup and refinement: Removed unused repository package, removed CSV persistence, eliminated dead code (duplicate main(), unused fields), and verified all imports.

7. Conclusion & Future Scope

Conclusion 7.1

The Vehicle Rental Management System successfully meets all stated requirements. It demonstrates a clean application of core OOP principles: inheritance and polymorphism through the Vehicle hierarchy, abstraction through the abstract calculateCost() contract, and encapsulation through controlled access to vehicle and rental state. The custom exception hierarchy provides typed, descriptive error reporting without leaking implementation details to the UI layer.

The Swing-based GUI provides a usable interface for the five main operations — fleet management, customer registration, rental processing, vehicle return, and status viewing — within a single window using CardLayout navigation. All operations are validated before committing state changes, ensuring the application remains consistent even when users supply invalid input.

Future Scope 7.2

- Persistent storage: Integrating a lightweight database such as SQLite would allow rental records, customer data, and fleet state to survive application restarts.
- Customer rental history: Extending the Customer class to hold a list of past Rental objects would enable per-customer rental reports.
- Date-based rentals: Replacing the integer days parameter with actual start and end dates would enable overdue tracking and late-return penalties.
- Search and filter: Adding search by vehicle type or availability in the Status panel would improve usability as the fleet grows.
- Role-based access: Distinguishing between administrator (fleet management) and staff (rental processing) roles with password protection.

- Pricing flexibility: Allowing per-vehicle rate overrides and promotional discounts without changing the domain classes.

Appendix

Initial Project Proposal A.

Project Proposal: Vehicle Rental Fleet Management System

Subtitle: A Modular, Multi-Tiered Approach to Fleet Management

1. Problem Statement

Manual vehicle rental management often leads to overbooking, inconsistent pricing, and poor data tracking. Modern systems require a way to handle diverse vehicle types (Cars, Trucks, Motorcycles) that each have unique maintenance needs and rental rules. The proposed Solution provides a robust OOP framework to automate availability tracking, calculate dynamic costs based on vehicle specifications, and ensure data integrity through structured error handling and organised storage.

2. Target User

Small to mid-sized rental agencies and corporate fleet managers who require a scalable, reliable desktop or backend solution.

3. Core Features

Fleet Management: Add, update, and categorize different vehicle types.

Dynamic Pricing: Automatic cost calculation based on vehicle type and rental duration.

Availability Tracking: Real-time status updates to prevent double-booking.

Transactional Safety: Custom exception handling for "Vehicle Not Found" or "Already Rented" scenarios.

4. OOP Concepts & Implementation

Concept	Application
Abstraction	Vehicle (Abstract Class) defines the core identity; IRepository (Interface) defines data contracts.
Inheritance	Car, Truck, and Motorcycle extend Vehicle, inheriting common fields
Polymorphism	The calculateCost(int days) method is overridden in subclasses to account for specific fees .
Encapsulation	State is protected via private fields and accessed through controlled public getters/setters.

5. System Architecture

The system follows a Layered Architecture to ensure modularity and clean separation of concerns.

I. Domain Layer (Entities)

Contains the core business objects. The Vehicle abstract class serves as the root, ensuring all vehicles have an ID and a model, while allowing subclasses to define their own "Rental Logic."

II. Service Layer (Logic)

The RentalManager acts as the orchestrator. It contains the business rules:

Checking if a customer is eligible.

Verifying vehicle availability.

Applying discounts or surcharges.

6. Error Handling Strategy

The system uses a hierarchy of Custom Checked Exceptions to ensure the UI can provide meaningful feedback to the user:

RentalException (Base)

VehicleNotFoundException (Thrown when searching for a non-existent ID)

AvailabilityException (Thrown if a vehicle is already booked)

InvalidDurationException (Thrown if rental days are <0)

7. Extensibility Consideration

The architecture is built for growth. One could easily add:

Discount Strategy: Using the Strategy Pattern to apply "Weekend Rates" or "Loyalty Discounts."

Review Mail (APPROVED_MINOR / MAJOR_REVISION) B.

PCCCS495 Proposal Review – APPROVED_MINOR

S soumadip.biswas@ieminternational.ai

To: APURBA MONDAL

Tue 3/10/2026 10:06 AM

Dear Apurba Mondal,

Project: Vehicle Rental Fleet Management System
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal demonstrates a strong understanding and application of core OOP principles, including Abstraction (Vehicle abstract class), Inheritance, Polymorphism (subclasses defining rental logic), Encapsulation, and Exception Handling (custom business-specific exceptions). The layered architecture (Domain, Service, Persistence) is well-defined and promotes excellent separation of concerns, modularity, and maintainability. The use of the Repository Pattern with an interface (`IVehicleRepository`) for data access abstraction is a particularly strong point, ensuring loose coupling and facilitating testability and future extensibility (e.g., swapping InMemory for SQL persistence). The problem statement is clear, and the proposed features directly address the identified pain points with a robust, OOP-centric solution.

Improvements:
While the proposal is excellent, a few areas could be enhanced. Firstly, explicitly defining additional core domain entities like `Customer` and `Rental` (representing the transaction) would round out the domain model, as the current focus is primarily on `Vehicle`. Secondly, clarify the exact scope of 'Rental Logic' within the `Vehicle` abstract class; ensure it primarily pertains to vehicle-specific attributes affecting cost or rules, reserving broader business rules (like customer eligibility or discount application) for the `RentalManager` in the Service Layer to maintain Single Responsibility. Lastly, 'organized storage' could be further detailed with examples or a brief mention of the storage technology (e.g., simple file storage, relational database).

Regards,
Course Instructor